

CELL 1: Install Dependencies

```
import sys
import subprocess

dependencies = [
    "gdown", "docling", "docling-core[chunking]",
    "sentence-transformers", "pinecone", "groq",
    "transformers", "torch", "rapidocr_onnxruntime", "accelerate"
]

class EnvironmentHalt(Exception):
    """Custom exception to cleanly halt IPython without triggering the
    ultratb framework bug."""
    def _render_traceback_(self):
        return ["\n[SYSTEM HALTED] -> Awaiting manual kernel
restart."]

def setup_environment():
    needs_restart = False
    try:
        import docling
        import pinecone
        import groq
        print("[System] Dependencies verified in RAM. Proceeding with
execution pipeline.")
    except ImportError:
        print("[System] Missing core dependencies. Initiating
installation sequence...")

        subprocess.run(
            [sys.executable, "-m", "pip", "install", "-qU"] +
dependencies,
            check=True
        )
        needs_restart = True

    # Decoupling the exit trigger from the exception handler prevents
    the IPython traceback crash
    if needs_restart:
        print("\n" + "="*60)
        print("CRITICAL ACTION REQUIRED:")
        print("1. The installation was successful.")
        print("2. You MUST restart the kernel to load the C-binaries
into RAM.")
        print("3. Click 'Run -> Restart Session' in the Kaggle
toolbar.")
        print("4. After restarting, click 'Run All' again.")
```

```
print("="*60 + "\n")
raise EnvironmentHalt()

setup_environment()

[System] Dependencies verified in RAM. Proceeding with execution
pipeline.
```

CELL 2: Core Library Imports

```
import os
import uuid
import torch
import gdown
from pathlib import Path
from kaggle_secrets import UserSecretsClient
from transformers import pipeline, AutoTokenizer
from sentence_transformers import SentenceTransformer
from pinecone import Pinecone, ServerlessSpec
from groq import Groq
from docling.document_converter import DocumentConverter,
PdfFormatOption
```

CELL 3: Hardware Verification & Secrets Management

```
# Detect CUDA (GPU) to accelerate PyTorch-based operations
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Executing pipeline on accelerator: {DEVICE.upper()}")

# Securely retrieve API keys from Kaggle's encrypted vault
secrets_client = UserSecretsClient()
PINECONE_API_KEY = secrets_client.get_secret("PINECONE_API_KEY")
GROQ_API_KEY = secrets_client.get_secret("GROQ_API_KEY")

# Expose Groq key to environment for the SDK to auto-detect
os.environ["GROQ_API_KEY"] = GROQ_API_KEY

Executing pipeline on accelerator: CUDA
```

CELL 4: Local Storage Provisioning & Repository Configuration

```
import time
from pathlib import Path

SESSION_ID = int(time.time())
RAW_DATA_DIR = Path(f"/tmp/pipeline_{SESSION_ID}/raw_pdfs")
MD_OUTPUT_DIR = Path(f"/tmp/pipeline_{SESSION_ID}/categorized_mds")

RAW_DATA_DIR.mkdir(parents=True, exist_ok=True)
MD_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

global_pdf_manifest = []

# Extract strictly the 33-character alphanumeric ID, NOT the entire URL
ZIP_FILE_ID = "luJ_WX23YXwEjuxal5-WUI43pGX0A0uDf"
```

CELL 5: Optimized Data Ingestion

```
import gdown
import zipfile
import traceback

zip_path = RAW_DATA_DIR / "compliance_payload.zip"

try:
    print("[NETWORK] Initiating block transfer for compressed archive...")
    # Pass the ID directly. Remove 'fuzzy' to satisfy the active function signature.
    downloaded = gdown.download(id=ZIP_FILE_ID, output=str(zip_path), quiet=False)

    if downloaded:
        print("[I/O] Decompressing binary payload into local memory...")
        with zipfile.ZipFile(zip_path, 'r') as zip_ref:
            zip_ref.extractall(RAW_DATA_DIR)

        pdfs = list(RAW_DATA_DIR.rglob("*.pdf"))
        global_pdf_manifest.extend(pdfs)
        print(f"[SUCCESS] Pipeline staged {len(global_pdf_manifest)} PDF artifacts.")
        zip_path.unlink()
    else:
```

```

        print("[FATAL] Network transfer returned a null payload.")
except Exception as e:
    print(f"[FATAL] Ingestion cascade collapsed. Reason: {e}")
    traceback.print_exc()

[NETWORK] Initiating block transfer for compressed archive...

Downloading...
From (original): https://drive.google.com/uc?id=1uJ_WX23YXwEjuxa15-
WUI43pGX0A0uDf
From (redirected): https://drive.google.com/uc?id=1uJ_WX23YXwEjuxa15-
WUI43pGX0A0uDf&confirm=t&uuid=9f3d1159-cfef-416b-b600-c21a32b85c72
To: /tmp/pipeline_1779985042/raw_pdfs/compliance_payload.zip
100%|██████████| 35.8M/35.8M [00:00<00:00, 245MB/s]

[I/O] Decompressing binary payload into local memory...
[SUCCESS] Pipeline staged 44 PDF artifacts.

```

CELL 6: Docling GPU Acceleration Setup

```

from docling.datamodel.base_models import InputFormat
from docling.datamodel.pipeline_options import PdfPipelineOptions
from docling.datamodel.accelerator_options import AcceleratorOptions,
AcceleratorDevice
import torch
from transformers import pipeline, AutoTokenizer
from sentence_transformers import SentenceTransformer
from docling.document_converter import DocumentConverter,
PdfFormatOption

# Define our processing namespaces
CATEGORIES = ["home_folder", "health_policy", "banking_governance",
              "regulatory_governance", "life_wealth_policy",
              "travel_policy", "vehicle_policy"]

# --- GPU 0 WORKLOAD: Layout Parsing & OCR ---
device_0 = torch.device("cuda:0" if torch.cuda.is_available() else
"cpu")

pipeline_options = PdfPipelineOptions()
pipeline_options.do_ocr = True # Enable advanced OCR fallback for
graphical PDFs
pipeline_options.do_table_structure = True # Enable structural table
extraction
pipeline_options.table_structure_options.do_cell_matching = True

# Bind the converter pipeline directly to GPU 0
doc_converter = DocumentConverter(

```

```

        format_options={
            InputFormat.PDF:
PdfFormatOption(pipeline_options=pipeline_options)
        }
    )

```

CELL 7: Zero-Shot Semantic Router Setup

```

# --- GPU 1 WORKLOAD: Classification & Embeddings ---
device_1_idx = 1 if torch.cuda.device_count() > 1 else 0

print(f"[SYSTEM] Sharding Layout Analysis to CUDA:0")
print(f"[SYSTEM] Sharding Inference & Embeddings to CUDA:
{device_1_idx}")

# Initialize Zero-Shot Classifier onto GPU 1
classifier = pipeline(
    "zero-shot-classification",
    model="facebook/bart-large-mnli",
    device=device_1_idx
)

# Initialize BGE-M3 Embedding Model onto GPU 1
embedding_model = SentenceTransformer("BAAI/bge-m3", device=f"cuda:
{device_1_idx}")
embedding_model.max_seq_length = 8192

[SYSTEM] Sharding Layout Analysis to CUDA:0
[SYSTEM] Sharding Inference & Embeddings to CUDA:1

{"model_id":"6bb59f57ee9149c69119958b1960da54","version_major":2,"vers
ion_minor":0}

{"model_id":"c9d863b1ee8140e7b304583440efd2d7","version_major":2,"vers
ion_minor":0}

```

CELL 8: High-Speed Batch OCR & Categorization

```

import gc
from pathlib import Path

# Ensure tracking directory structure is clean
categorized_md_map = {cat: [] for cat in CATEGORIES}

```

```

print(f"[START] Processing {len(global_pdf_manifest)} files in
queue...")

for pdf_path in global_pdf_manifest:
    try:
        # Step 1: Layout Parsing & Table Structure on GPU 0
        conversion_result = doc_converter.convert(str(pdf_path))
        md_text = conversion_result.document.export_to_markdown()

        if not md_text.strip():
            print(f"[WARNING] Skipping empty file: {pdf_path.name}")
            continue

        # Step 2: Content Sequence Classification on GPU 1
        # Truncate input to 2000 chars to speed up routing inference
        classification_payload = classifier(md_text[:2000],
CATEGORIES)
        assigned_cat = classification_payload["labels"][0]

        # Step 3: Write structural markdown to the dynamic disk path
        target_dir = MD_OUTPUT_DIR / assigned_cat
        target_dir.mkdir(parents=True, exist_ok=True)

        output_md_path = target_dir / f"{pdf_path.stem}.md"
        with open(output_md_path, "w", encoding="utf-8") as f:
            f.write(md_text)

        categorized_md_map[assigned_cat].append(output_md_path)
        print(f"[SUCCESS] {pdf_path.name} -> Classified as:
{assigned_cat.upper()}")

    except Exception as e:
        print(f"[ERROR] Failed to process document {pdf_path.name}.
Log: {e}")
        continue

# Flush temporary allocation caches
torch.cuda.empty_cache()
gc.collect()

print("\n[FINISH] Document processing stage complete. Arrays mapped to
target directories.")

[INFO] 2026-05-28 16:17:31,347 [RapidOCR] base.py:22: Using
engine_name: onnxruntime
[INFO] 2026-05-28 16:17:31,362 [RapidOCR] download_file.py:60: File
exists and is valid:
/usr/local/lib/python3.12/dist-packages/rapidocr/models/ch_PP-
OCrv4_det_mobile.onnx
[INFO] 2026-05-28 16:17:31,362 [RapidOCR] main.py:57: Using

```

```
/usr/local/lib/python3.12/dist-packages/rapidocr/models/ch_PP-OCRv4_det_mobile.onnx
[INFO] 2026-05-28 16:17:31,487 [RapidOCR] base.py:22: Using
engine_name: onnxruntime
[INFO] 2026-05-28 16:17:31,490 [RapidOCR] download_file.py:60: File
exists and is valid:
/usr/local/lib/python3.12/dist-packages/rapidocr/models/ch_ppocr_mobil
e_v2.0_cls_mobile.onnx
[INFO] 2026-05-28 16:17:31,490 [RapidOCR] main.py:57: Using
/usr/local/lib/python3.12/dist-packages/rapidocr/models/ch_ppocr_mobil
e_v2.0_cls_mobile.onnx
```

```
[START] Processing 44 files in queue...
```

```
[INFO] 2026-05-28 16:17:31,558 [RapidOCR] base.py:22: Using
engine_name: onnxruntime
[INFO] 2026-05-28 16:17:31,594 [RapidOCR] download_file.py:60: File
exists and is valid:
/usr/local/lib/python3.12/dist-packages/rapidocr/models/ch_PP-
OCRv4_rec_mobile.onnx
[INFO] 2026-05-28 16:17:31,595 [RapidOCR] main.py:57: Using
/usr/local/lib/python3.12/dist-packages/rapidocr/models/ch_PP-
OCRv4_rec_mobile.onnx
```

```
{"model_id": "dc987ad61ed949c089bf6b250ce3ff58", "version_major": 2, "vers
ion_minor": 0}
```

```
[SUCCESS] CANARA BANK1.pdf -> Classified as: HOME_FOLDER
```

```
RapidOCR returned empty result!
RapidOCR returned empty result!
RapidOCR returned empty result!
RapidOCR returned empty result!
```

```
[SUCCESS] bob2 (1).pdf -> Classified as: REGULATORY_GOVERNANCE
[SUCCESS] indusind5.pdf -> Classified as: LIFE_WEALTH_POLICY
[SUCCESS] irdai3 (1).pdf -> Classified as: REGULATORY_GOVERNANCE
[SUCCESS] irdai10.pdf -> Classified as: HOME_FOLDER
[SUCCESS] icici5.pdf -> Classified as: HEALTH_POLICY
[SUCCESS] bob1.pdf -> Classified as: HEALTH_POLICY
```

```
[WARNING] 2026-05-28 16:23:26,239 [RapidOCR] main.py:132: The text
detection result is empty
RapidOCR returned empty result!
[WARNING] 2026-05-28 16:23:28,681 [RapidOCR] main.py:132: The text
detection result is empty
RapidOCR returned empty result!
[WARNING] 2026-05-28 16:23:30,244 [RapidOCR] main.py:132: The text
detection result is empty
RapidOCR returned empty result!
[WARNING] 2026-05-28 16:23:31,840 [RapidOCR] main.py:132: The text
```

detection result is empty
RapidOCR returned empty result!
[WARNING] 2026-05-28 16:23:33,378 [RapidOCR] main.py:132: The text
detection result is empty
RapidOCR returned empty result!

[SUCCESS] icici6.pdf -> Classified as: HOME_FOLDER
[SUCCESS] icici3 (1).pdf -> Classified as: REGULATORY_GOVERNANCE
[SUCCESS] icici4 (1).pdf -> Classified as: HEALTH_POLICY

[WARNING] 2026-05-28 16:26:51,676 [RapidOCR] main.py:132: The text
detection result is empty
RapidOCR returned empty result!

[SUCCESS] hdfc2.pdf -> Classified as: HEALTH_POLICY
[SUCCESS] SBI3.pdf -> Classified as: HOME_FOLDER
[SUCCESS] kotak_mah3.pdf -> Classified as: HEALTH_POLICY
[SUCCESS] irdai9.pdf -> Classified as: REGULATORY_GOVERNANCE
[SUCCESS] irdai8.pdf -> Classified as: REGULATORY_GOVERNANCE
[SUCCESS] irdai4.pdf -> Classified as: HOME_FOLDER
[SUCCESS] irdai5.pdf -> Classified as: REGULATORY_GOVERNANCE
[SUCCESS] irdai2.pdf -> Classified as: REGULATORY_GOVERNANCE
[SUCCESS] irdai1.pdf -> Classified as: REGULATORY_GOVERNANCE
[SUCCESS] SBI1.pdf -> Classified as: VEHICLE_POLICY
[SUCCESS] axis3.pdf -> Classified as: VEHICLE_POLICY
[SUCCESS] kotak_mah1.pdf -> Classified as: TRAVEL_POLICY
[SUCCESS] axis2.pdf -> Classified as: TRAVEL_POLICY
[SUCCESS] SBI2.pdf -> Classified as: HEALTH_POLICY
[SUCCESS] CANARA BANK4.pdf -> Classified as: HEALTH_POLICY
[SUCCESS] hdfc4.pdf -> Classified as: HOME_FOLDER
[SUCCESS] bob3.pdf -> Classified as: VEHICLE_POLICY
[SUCCESS] axis1.pdf -> Classified as: HEALTH_POLICY
[SUCCESS] icici1 (1).pdf -> Classified as: VEHICLE_POLICY
[SUCCESS] hdfc1.pdf -> Classified as: VEHICLE_POLICY
[SUCCESS] icici2 (2).pdf -> Classified as: HEALTH_POLICY
[SUCCESS] irdai7 (1).pdf -> Classified as: HEALTH_POLICY
[SUCCESS] kotak_mah3.pdf -> Classified as: HEALTH_POLICY
[SUCCESS] indusind1 (1).pdf -> Classified as: HOME_FOLDER
[SUCCESS] kotak_mah2.pdf -> Classified as: REGULATORY_GOVERNANCE
[SUCCESS] indusind4.pdf -> Classified as: HEALTH_POLICY
[SUCCESS] CANARA BANK1.pdf -> Classified as: HOME_FOLDER
[SUCCESS] CANARA BANK2.pdf -> Classified as: BANKING_GOVERNANCE
[SUCCESS] federal 4.pdf -> Classified as: BANKING_GOVERNANCE
[SUCCESS] federal 2.pdf -> Classified as: VEHICLE_POLICY
[SUCCESS] federal 1.pdf -> Classified as: VEHICLE_POLICY
[SUCCESS] Indusind2.pdf -> Classified as: BANKING_GOVERNANCE
[SUCCESS] federal6.pdf -> Classified as: BANKING_GOVERNANCE

```
[WARNING] 2026-05-28 16:46:35,809 [RapidOCR] main.py:132: The text
detection result is empty
RapidOCR returned empty result!

[SUCCESS] federal 3.pdf -> Classified as: REGULATORY_GOVERNANCE

[FINISH] Document processing stage complete. Arrays mapped to target
directories.
```

CELL 9: Token-Aware Chunker & Embedding Initialization

```
from docling_core.transforms.chunker.hybrid_chunker import
HybridChunker
from docling_core.transforms.chunker.tokenizer.huggingface import
HuggingFaceTokenizer
from docling.document_converter import DocumentConverter

EMBED_MODEL = "BAAI/bge-m3"

# Tokenizer mapped to BGE-M3 limits (Model is already safely loaded on
GPU 1 from Cell 7)
hf_tokenizer =
HuggingFaceTokenizer(tokenizer=AutoTokenizer.from_pretrained(EMBED_MOD
EL))
chunker = HybridChunker(tokenizer=hf_tokenizer, max_tokens=1024,
merge_peers=True)

# Lightweight parser specifically for the markdown files (defaults to
CPU, minimal footprint)
md_parser = DocumentConverter()
```

CELL 10: Pinecone Serverless Provisioning

```
import time
from pinecone import Pinecone, ServerlessSpec

# Connect to Pinecone backend
pc = Pinecone(api_key=PINECONE_API_KEY)
INDEX_NAME = "compliance-rag-index"

# Provision a serverless index matching BGE-M3's exact 1024 dimensions
if INDEX_NAME not in [idx.name for idx in pc.list_indexes()]:
    print(f"[NETWORK] Provisioning Serverless Index: {INDEX_NAME}...")
    pc.create_index(
```

```

        name=INDEX_NAME,
        dimension=1024,
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )

    # Asynchronous Readiness Lock: Wait for cloud hardware allocation
    while not pc.describe_index(INDEX_NAME).status['ready']:
        print("[NETWORK] Awaiting AWS backend initialization...
sleeping 5s")
        time.sleep(5)
        print(f"[SUCCESS] Index {INDEX_NAME} is fully online and accepting
connections.")

# Connect to the fully initialized remote index
index = pc.Index(INDEX_NAME)

[NETWORK] Provisioning Serverless Index: compliance-rag-index...
[SUCCESS] Index compliance-rag-index is fully online and accepting
connections.

```

CELL 11: Chunking and Batch Upsertion to Pinecone

```

import gc

PINECONE_BATCH_SIZE = 75

for category, md_files in categorized_md_map.items():
    ns_vectors = []
    for md_file in md_files:
        chunks =
list(chunker.chunk(dl_doc=md_parser.convert(str(md_file)).document))
        texts = [c.text for c in chunks if c.text.strip()]
        if not texts: continue

        # THROTTLE VRAM ALLOCATION: Set encode batch_size aggressively
low to prevent attention matrix spikes
        embs = embedding_model.encode(texts, batch_size=8,
convert_to_tensor=False)

        for chunk_obj, emb in zip(chunks, embs):
            ns_vectors.append({
                "id": uuid.uuid4().hex,
                "values": emb.tolist(),
                "metadata": {"source": md_file.name, "text":
chunk_obj.text}

```

```

    })

    for i in range(0, len(ns_vectors), PINECONE_BATCH_SIZE):
        index.upsert(vectors=ns_vectors[i:i + PINECONE_BATCH_SIZE],
                     namespace=category)

        # GARBAGE COLLECTION: Purge orphaned tensors from VRAM after
processing each category
        torch.cuda.empty_cache()
        gc.collect()

/usr/local/lib/python3.12/dist-packages/docling/backend/
md_backend.py:108: UserWarning: Detected potentially incorrect
Markdown, correcting...
    warnings.warn("Detected potentially incorrect Markdown,
correcting...")
/usr/local/lib/python3.12/dist-packages/docling/backend/md_backend.py:
108: UserWarning: Detected potentially incorrect Markdown,
correcting...
    warnings.warn("Detected potentially incorrect Markdown,
correcting...")
/usr/local/lib/python3.12/dist-packages/docling/backend/md_backend.py:
108: UserWarning: Detected potentially incorrect Markdown,
correcting...
    warnings.warn("Detected potentially incorrect Markdown,
correcting...")
[transformers] Token indices sequence length is longer than the
specified maximum sequence length for this model (8327 > 8192).
Running this sequence through the model will result in indexing errors
/usr/local/lib/python3.12/dist-packages/docling/backend/md_backend.py:
108: UserWarning: Detected potentially incorrect Markdown,
correcting...
    warnings.warn("Detected potentially incorrect Markdown,
correcting...")

```

CELL 12: System Prompt and Groq Client Initialization

```

# SDK automatically utilizes os.environ
groq_client = Groq()

# Highly constrained system prompt targeting heavy legal compliance
tasks
SYS_PROMPT = """
You are a high-speed Regulatory Auditor AI.
Use strictly the retrieved evidence context blocks to answer the
query.

```

Mandatory: End your response with a citation of the source file name.
If the answer isn't in the provided text, reply exactly with:
"Insufficient compliance data."
"""

CELL 13: Low-Latency RAG Query Pipeline

```
def query_rag(question: str, target_namespace: str):
    query_vec = embedding_model.encode(question).tolist()

    # Throttle network retrieval: reduce top_k from 3 to 2 to minimize
    # initial bloat
    results = index.query(namespace=target_namespace,
                          vector=query_vec, top_k=4, include_metadata=True)

    context = "\n\n".join([f"Source: {m['metadata']['source']}\nText: {m['metadata']['text']}" for m in results['matches']])

    # HARD TRUNCATION (The Circuit Breaker):
    # Forcefully slice the context to a maximum of 15,000 characters
    # (~3,750 tokens).
    # This mathematically guarantees you stay beneath Groq's 6,000 TPM
    # limit.
    safe_context = context[:15000]

    messages = [
        {"role": "system", "content": SYS_PROMPT},
        {"role": "user", "content": f"Context Evidence:\n{n{safe_context}}\n\nQuery: {question}"}
    ]

    # We maintain temperature=0.01 for deterministic, zero-
    # hallucination compliance answers
    response = groq_client.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=messages,
        temperature=0.02
    )

    print(f"[{target_namespace.upper()}]\nQ: {question}\nA: {response.choices[0].message.content}\n")
```

CELL 14: Automated Verification Tests

```
# Properly initialized array of tuples
test_queries = [
```

```
    ("What are the core requirements for cross-border data transfer?",  
    "regulatory_governance"),  
    ("What is the maximum penalty for health insurances for data non-  
compliance?", "health_policy")  
]
```

```
for question, namespace in test_queries:  
    query_rag(question, namespace)
```

[REGULATORY_GOVERNANCE]

Q: What are the core requirements for cross-border data transfer?

A: Insufficient compliance data.

However, based on the provided context, it seems that the core requirements for cross-border data transfer are not explicitly mentioned. But, in the context of insurance regulations, it can be inferred that the requirements for cross-border data transfer might be related to the protection of policyholders' interests, compliance with applicable laws, and regulatory approvals.

For instance, in the context of the IRDAI (Insurance Regulatory and Development Authority of India) regulations, the requirements for cross-border data transfer might be related to the submission of confirmations by shareholders, protection of interests of policyholders, creditors, and/or shareholders, and compliance with the Regulations/Circulars on Corporate Governance issued by the Authority.

However, a more specific answer cannot be provided without further information on cross-border data transfer requirements.

Source: irdail.md

[HEALTH_POLICY]

Q: What is the maximum penalty for health insurances for data non-compliance?

A: Insufficient compliance data.

However, I can provide some information from the text. According to the text, in case the Insurer does not honour the ombudsman award, a penalty of Rs. 5000/- per day shall be payable to the complainant.